

Moss

Julia-Like Ergonomics with Static Safety and Compiler-Derived Concurrency

Language Design, Formal Safety Model, and v0.1 Implementation Experience

Kutty Banerjee

September 2026

Moss v0.1 thesis. Write code with the economy of a high-level language; close types, call targets, ownership effects, domain topology, and synchronization statically; lower the result to direct safe Rust. The compiler, not the programmer, chooses Moss-managed locks and their order.

Abstract

Moss is a compiled language designed to feel closer to Julia than to a traditional systems language while still producing a statically closed native program. Programmers can omit many type annotations, use structural traits without writing `implements` declarations, and rely on specialization instead of runtime dynamic dispatch. Before native code generation, however, Moss closes the concrete types, call targets, ownership effects, domain topology, and synchronization requirements of the reachable program. Moss compiles to safe Rust, deliberately reusing Rust’s mature memory-safety discipline and native backend while restricting selected concurrency patterns further so that lock choice, lock ownership, and Moss-managed lock order become compiler facts rather than programmer decisions.

The central concurrency abstraction is the *domain*: a statically composed owner of mutable state entered only through synchronous `message`. Programmers write neither mutexes nor atomics. For each concrete handler, the compiler infers `READ/WRITE/CONSUME` effects, derives the protected mutable state, partitions that state into synchronization classes, and emits direct typed `RwLock<ClassState>` fields. Handlers acquire exactly their statically known classes in rank order and hold them through the complete handler, including nested messages. This supports a structural deadlock-freedom argument for Moss-managed locks and domain-local conflict serializability for protected state.

The paper separates four concerns. Part I presents Moss as a language: “infer all the way, then close statically,” structural traits, ownership effects, domains, synchronous messaging, the programmer-visible consistency model, and features that make the language unusually tractable for AI-assisted and agentic coding. Part II states the formal model, including observable non-domain effects, conservative path-insensitive effect inference, synchronization classes, static-footprint strict two-phase locking, global lock ranking, and proof assumptions. Part III describes the v0.1 implementation and its design corrections, including the replacement of mailbox/`await` semantics, closure of the domain-handle universe, zero-copy protected reads, and the final static typed lowering that reduced lock-wrapper overhead to the same cost class as equivalent handwritten Rust. Part IV places Moss against prior lock inference, effect systems, actors, DPJ, Pony, Julia, and Rust, and states the limits of the current claims.

Contents

I	The Language	2
1	The simple pitch: write high-level code, compile a closed systems program	2
2	A small surface with structural specialization	2
3	Ownership effects are inferred, not spelled with parameter modifiers	3
4	Functional/dataflow code without general closure objects	3
5	Domains: state ownership as an architectural construct	4
6	Static composition and immutable routing	4
7	Synchronous message and terminating reply	5
8	Programmer-visible consistency and ordering	5
9	By-value domain boundaries	6
10	Observable effects beyond domain state	6
11	Safety by restriction: Rust strengths plus additional static concurrency constraints	6

12	Diagnostics are part of the language product	7
13	Modules and semantic interfaces	8
14	One frontend, two execution engines	8
15	Features for AI-assisted and agentic coding	8
16	Intentional v0.1 boundaries	10
17	Future domain scopes: an intentionally open breadcrumb	10
II	Formal Model and Safety Arguments	11
18	Static objects of the model	11
19	State effects and observable effects	11
20	Mutable universe and protected reads	11
21	Synchronization modes and class partition	12
21.1	Class count is a cost as well as an opportunity	12
22	Running Account derivation	12
23	Path-insensitive locking: the cache-populate bound	13
24	Static-footprint strict two-phase locking	13
25	Global lock rank and deadlock freedom	14
26	Why the concrete graph must be closed	14
27	Publication and immutable reads	14
28	Ownership, aliases, and domain boundaries	15
29	Failure model and proof scope	15
30	Claims and non-claims	15
III	Implementation Design, Corrections, and Measurements	16
31	Compilation pipeline	16
32	Final production lowering: locks own their protected state	16
33	Why the first generic runtime was rejected	17
34	Borrowed protected reads	17
35	From asynchronous actors to synchronous domains	17

36 Closing the domain-handle universe	18
37 Fast Debug as a semantic engine	18
38 Performance validation: codegen overhead, not a scalability claim	18
38.1 What the concurrency measurements do not prove	18
39 Class count, nested hold time, and other measured costs	19
40 Modules: semantic ABI, physical lowering at the consumer	19
41 Reachability and other conservative bounds	19
42 Design evolution summary	19
43 Dogfooding before expansion	20
 IV Related Work and Positioning	 20
44 Synchronization inference and lock allocation	20
45 Effect systems and Deterministic Parallel Java	20
46 Actors and capability-based race freedom	21
47 Rust and Julia as complementary influences	21
48 Open questions	21
49 Conclusion	21
A Compact v0.1 semantic summary	22
B Proof assumptions checklist	22
C Moss v0.1 milestone map	23
D References	23

The Language

1 The simple pitch: write high-level code, compile a closed systems program

The simplest way to describe Moss is: **write it like a high-level language, compile it like a systems language**. A Moss function can look Julia-like—few annotations, structural constraints instead of nominal conformance declarations, and specialization based on concrete use—but the compiler refuses to leave dynamic uncertainty unresolved at native code generation. Moss attempts to infer all the way down: types, call targets, ownership effects, concrete domain identities, routing, and synchronization are derived when the program provides enough information; unresolved behavior is rejected before the final native program is emitted.

Moss is not Julia. Julia intentionally supports dynamic dispatch and runtime method selection; Moss v0.1 deliberately does not. The resemblance is ergonomic rather than semantic: source should describe the computation rather than repeat facts the compiler can infer. The compiled result is closer in spirit to a fully monomorphized systems program.

Moss is also not a replacement for Rust’s safety model. It compiles to safe Rust and leverages Rust’s ownership-oriented type system, standard library, native code generation, and tooling strengths. Moss narrows the source language further in selected areas. In particular, programmers do not select Moss-managed locks, do not write lock order, cannot dynamically rewire domain references, and cannot pass domain handles as ordinary values. The resulting claim is deliberately scoped: *within the Moss domain model, additional concurrency mistakes can be ruled out by construction because the relevant choices are absent from the language surface*.

Key idea. Moss’s design principle is **infer all the way, then close statically**. Source code may remain locally under-specified; the native program may not.

2 A small surface with structural specialization

Moss supports primitives, value-oriented objects, aggregates, collections, ordinary functions, traits, modules, functional/dataflow operators, and domains. Not every function boundary must spell out a concrete type. Untyped parameters are statically specialized at concrete uses.

Traits are structural compile-time predicates rather than nominal memberships. A type does not say that it implements a trait; conformance is checked when a concrete type reaches a use that requires the trait.

```
trait Drawable:
  fn area() -> Int
  fn draw(canvas: Canvas) -> Int

type Circle:
  radius: Int

  fn area():
```

```

    return radius * radius

fn draw(canvas: Canvas):
    return radius * radius * canvas.scale

fn render(x: Drawable, canvas: Canvas):
    return x.draw(canvas)

```

A `Circle` satisfies `Drawable` because the required operations match. No `implements Drawable` declaration is needed. A type may therefore satisfy many traits without being coupled to their declarations.

Moss v0.1 does not have runtime trait objects or general dynamic dispatch. Structural conformance is resolved during specialization. This is important to the larger thesis: the language can feel duck-typed locally while still closing statically before native code generation.

3 Ownership effects are inferred, not spelled with parameter modifiers

Moss uses three semantic access effects:

Effect	Meaning
read	May observe the caller's storage/value.
write	May mutate caller-visible storage.
consume	May take ownership of the current value under Moss ownership rules.

Effects are inferred from the specialized body and propagated through ordinary calls. There is no planned `mut`, `inout`, or `write` parameter modifier merely to communicate these facts to the compiler.

At a call site, the compiler checks capabilities. `READ` requires readable storage/value; `WRITE` requires legal mutable caller storage; `CONSUME` requires ownership/move capability. Overlapping actual arguments are accepted for `READ+READ` and rejected whenever overlap involves `WRITE` or `CONSUME`. A literal or temporary cannot satisfy a caller-visible `WRITE` formal.

This design keeps source lightweight while retaining static closure. It also makes synchronization derivation possible later: transitive writes through helpers are visible to the handler effect summary even when the handler body does not contain the mutation syntactically.

4 Functional/dataflow code without general closure objects

Moss supports concise functional/dataflow forms such as `map`, `filter`, `reduce`, `sum`, `count`, `any`, and `all`, including restricted placeholder/capture forms. Static callable specialization occurs before effect derivation, and captures of domain state contribute to handler effects.

Moss v0.1 does **not** provide general first-class closure values or runtime higher-order dispatch. A bound callable that participates in a functional pipeline is statically resolved. This distinction matters because the implementation can reason about a finite set of concrete callable bodies while still offering concise dataflow syntax.

Ordinary recursion is also disallowed in v0.1, but not for the reason that synchronization-effect closure would be impossible. With a finite effect lattice, recursive call-graph SCCs can be solved by a least fixed point. The v0.1 restriction is instead an implementation/language-surface choice: recursion raises separate questions about specialization termination, inference ergonomics, diagnostics, and unbounded stack depth. The synchronization proof does not require ordinary recursion to remain forbidden. Domain-level recursive call structure is different: the concrete domain routing graph is required to be acyclic.

This distinction is important for dogfooding. Tree walks, parsers, compiler passes, and other naturally recursive programs may provide early evidence that ordinary recursion deserves a later phase.

5 Domains: state ownership as an architectural construct

A domain owns mutable state and exposes handlers as the only externally callable operations on that state. A domain is not defined by a mailbox or worker thread. In the final v0.1 model, a domain is a static ownership and synchronization boundary.

The paper uses one running example throughout:

```
domain Account:
  balance: Int
  risk_limit: Int
  display_name: String
  stats: Stats
  config_value: Int

  on rename(name):
    display_name = name

  on set_risk_limit(limit):
    risk_limit = limit

  on record_fill(fill):
    balance = balance + fill.amount
    stats = update_stats(stats, fill)
    validate_risk(balance, risk_limit)

  on read_stats():
    reply stats

  on read_config():
    reply config_value
```

The programmer did not write a lock. The compiler will eventually discover that `config_value` is immutable after publication, while the other fields participate in different synchronization classes.

6 Static composition and immutable routing

Domains are wired in an initial composition prefix. A domain declares outbound routing dependencies with `domainroutes(...)`. Domain handles are routing capabilities, not ordinary values.

```
domain App:
  domainroutes(account: Account, logger: Logger)

  on run():
    receipt = message account.read_stats()
    message logger.record(receipt)

fn main(config):
  account = Account(
    balance = config.start_balance,
    risk_limit = config.limit,
```

```
display_name = config.name,  
stats = initial_stats(),  
config_value = config.mode)  
  
logger = Logger(...)  
app = App(account = account, logger = logger)  
  
message app.run()
```

The number of concrete domain instances, their identities, and every route binding are statically enumerable. Initialization values may depend on runtime configuration through pure expressions; topology may not depend on runtime control flow.

Domain handles cannot be passed as ordinary function parameters, handler payloads, reply values, collection elements, mutable state, or aliases. Every legal cross-domain message target is therefore attributable to the closed concrete domain graph.

Terminology note

Chapel also uses the term *domain*, but for a different language construct: Chapel domains are first-class index sets used to define array index spaces. Moss domains are state-owning architectural/synchronization units. The shared word should not be read as a semantic relationship.

7 Synchronous message and terminating reply

`message` is the only operation that enters a domain handler. It is synchronous, blocking, and expression-valued:

```
stats = message account.read_stats()  
message logger.record(stats)
```

There is no domain-level `await` and no fire-and-forget send in v0.1. A message in statement position simply discards the result; it remains synchronous.

`reply` is terminating. Its expression is evaluated while the handler's synchronization is still held; an independent semantic result is established; the handler terminates; its locks are released; then the caller resumes. No statement after a taken reply executes.

Incoming message payloads are immutable snapshots. Handlers may read them, forward them through another message (creating a new value boundary), or reply with them (again creating a new value boundary), but may not mutate or consume the original incoming snapshot.

8 Programmer-visible consistency and ordering

Moss intentionally does **not** promise sequential consistency across the entire domain graph.

Within one domain, failure-free completed handler executions are conflict-serializable with respect to that domain's protected mutable state. Conflicting handlers serialize; handlers whose synchronization footprints are compatible may overlap. Synchronous calls impose ordinary source order along a call chain: if handler A calls B and waits for its reply, B completes before A continues.

Across independent domains, however, there is no single global order of all observable actions. Two handlers executing concurrently in different domains may produce effects that downstream observers see in different

orders unless the program establishes an ordering through synchronous dependencies. Moss therefore provides domain-local conflict serializability and explicit synchronous program order, not graph-wide sequential consistency or a multi-domain transaction.

This point is part of the language model, not merely a non-claim hidden in the proof section.

9 By-value domain boundaries

Message arguments and reply results are semantic by-value boundaries. The programmer can rely on independence even if the backend implements the boundary with a move, return-value optimization, storage reuse, or another copy-elision technique.

A Rust reference into one domain’s state never becomes another domain’s message payload. Callables themselves do not cross message or reply. This keeps the cross-domain boundary first-order and prevents dynamic route/callable capability propagation.

10 Observable effects beyond domain state

Synchronization effects and all observable effects are not the same thing. Moss derives READ/WRITE/CONSUME over domain-owned state because those effects drive synchronization. We separately write $O(h)$ for the observable non-domain-state effects of handler h : output such as `echo`, and in future versions foreign I/O or system effects crossing an interoperability boundary.

The v0.1 lock planner does not partition on $O(h)$. Such effects occur while the handler’s locks are held because locks span the complete handler. Synchronous source order is preserved within one execution chain, but Moss does not impose a total order on $O(h)$ across independent concurrently executing domains.

Making $O(h)$ explicit prevents a common misunderstanding: domain-state serializability is not a claim that every externally visible effect in the application participates in one transaction order.

11 Safety by restriction: Rust strengths plus additional static concurrency constraints

Moss compiles to safe Rust and relies on Rust’s memory-safety and data-race guarantees in safe code. The contribution is not that Moss somehow makes Rust’s existing guarantees conditional or stronger in every dimension. Rather, Moss removes additional concurrency choices from normal source code.

Concern	Safe Rust	Moss v0.1
Memory/ownership	Ownership and borrowing prevent broad classes of memory errors	Compiles to safe Rust plus inferred Moss ownership/capability checks
Data races	Prevented in safe Rust by the type system and safe synchronization/interior-mutability abstractions	Also prevented; Moss additionally derives the synchronization objects and access modes for domain state
Forgotten/wrong lock	Possible logic error in otherwise safe Rust	No user lock selection exists for Moss-managed domain state
Lock-order deadlock	Possible	Ruled out for Moss-managed domain locks under the static graph/rank assumptions
Dynamic route/lock graph	Fully programmable	Concrete domain routing graph is statically closed and acyclic
Dispatch	Static and dynamic mechanisms available	No runtime dynamic dispatch in v0.1
Recursion	Allowed	Ordinary recursion is a v0.1 restriction for specialization/implementation reasons, not the lock proof
Concurrency topology	General-purpose	Deliberately restricted static domain topology

Table 1: Moss trades generality for compiler ownership of additional concurrency decisions.

The useful edge over Rust is therefore concentrated in the middle of the table: a programmer cannot forget the Moss-managed lock, choose the wrong one, or establish an inconsistent Moss lock order, because those operations are not part of the Moss source language.

12 Diagnostics are part of the language product

A language that removes programmer-written locks must explain the restrictions and rewrites that make the proof possible. Representative diagnostics should be concrete and teach the model.

```
error: same-domain handler entry is not permitted
  message account.refresh()
    ~~~~~
'account' is the currently executing domain instance.

rewrite: extract shared same-domain logic into an ordinary helper
```

```
error: domain handle cannot cross a message boundary
  message worker.run(database)
    ~~~~~
'database' is a domain routing capability, not a value payload.

rewrite: declare an immutable domainroutes dependency on Worker
```

```
error: concrete domain routing cycle
  app  --cache--> cache
  cache --store--> store
  store --owner--> app

Moss requires the concrete domain routing graph to be acyclic.
```

These are not merely nicer error messages. They are the mechanism by which a restricted safety model

remains usable.

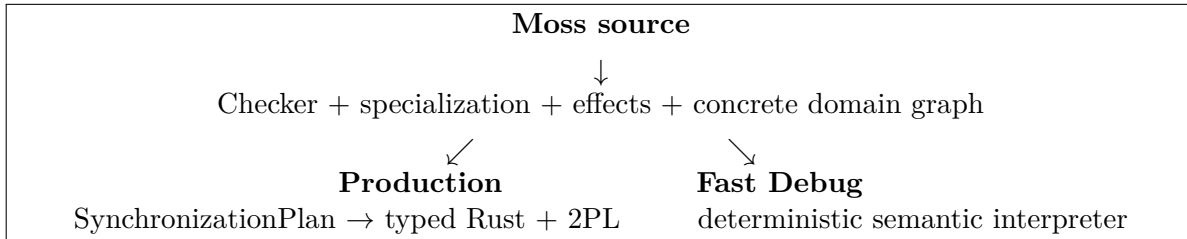
13 Modules and semantic interfaces

Moss v0.1 supports explicit modules, project builds, typed exports, semantic interfaces, and source-free production providers. The module system is usable but intentionally not declared final; dogfooding is expected to reveal where import ergonomics, package discovery, and cross-module specialization need improvement.

The key separation is semantic versus physical ABI. A provider can export checked type/effect information sufficient for downstream specialization. Physical synchronization classes, lock ranks, Rust lifetimes, and generated lock layout are not module ABI. The final application derives them from its concrete graph and emits physical synchronization itself.

14 One frontend, two execution engines

Moss has one checked semantic frontend and two execution engines:



Production assumes concurrent handler entry may occur and therefore uses physical synchronization. Fast Debug executes the same checked domain semantics directly and deterministically without simulating locks, threads, or alternative schedules. Its structured trace exposes source identity, concrete instance identity, handler entry/exit, state reads/writes, messages, replies, and branches.

15 Features for AI-assisted and agentic coding

Moss is not designed around the premise that an AI agent should be trusted to generate more low-level machinery. The opposite is more useful: the language removes choices that are expensive for both humans and agents to get right, then exposes the compiler’s semantic knowledge in forms an agent can inspect. The result is a smaller legal program space, a deterministic debug loop, and fewer multi-step stateful interactions with external debuggers.

This matters because agentic coding has a different failure profile from ordinary interactive programming. An agent can generate plausible code quickly, but it is vulnerable to hidden dynamic behavior, tool-state drift, ambiguous diagnostics, and long sequences of debugger commands whose intermediate state must be remembered correctly. Moss’s restrictions and tooling reduce those sources of uncertainty without changing the semantic contract for human programmers.

Feature	Why it helps an AI coding agent	v0.1 status
Static closure	Concrete call targets, domain instances, routes, effects, and synchronization are resolved before native code generation. The agent reasons about a finite checked program rather than an open runtime dispatch space.	Implemented
Restricted concurrency surface	The agent does not invent locks, lock order, actor sharding, or dynamic routing schemes for ordinary Moss domain state. Those decisions are compiler-derived.	Implemented
Structural traits and inference	Agents can write concise code without manufacturing nominal conformance declarations or redundant generic type variables merely to satisfy the compiler. Errors are discovered when concrete uses fail to specialize.	Implemented
Stable semantic identities	Source entities and concrete specializations have stable compiler identities/provenance that can be surfaced to tools instead of reconstructed from textual guesses.	Implemented
Semantic introspection	The checked compiler model can answer machine-oriented questions about specializations, effects, concrete domain topology, routes, and synchronization plans.	Implemented
Deterministic Fast Debug	An agent can execute checked Moss semantically without managing a native debugger session, thread schedule, lock timing, or process-control state.	Implemented
Structured execution trace	Handler entry/exit, branches, messages, replies, and state accesses can be represented as structured events suitable for search, slicing, comparison, and model context.	Implemented base-line
Machine-teaching diagnostics	Restrictions can be reported in terms of the offending construct, the language rule, and a legal Moss rewrite, turning compiler failures into a teaching channel for agents.	Future Phase 22 expansion
Agent benchmark suite	Representative tasks can measure first-attempt compile rate, iterations-to-green, error classes, partition quality, and context efficiency instead of relying on anecdotes.	Future Phase 22

Table 2: Moss properties that reduce ambiguity and tool-state burden for AI-assisted programming.

A deterministic debug loop instead of an interactive debugger protocol

Traditional native debugging is often an interaction protocol: set a breakpoint, run, inspect, step, print, continue, and repeat. For an agent, each step is another tool call and another piece of ephemeral process state that must remain synchronized with its reasoning. Moss’s Fast Debug engine converts many small-scope debugging problems into a data-analysis problem: run the checked program deterministically, emit a structured trace, and reason over the resulting execution record.

The intended agent loop is therefore closer to:

edit Moss source → check/specialize → run Fast Debug → inspect diagnostics + trace → edit

rather than “drive LLDB correctly for twenty turns.” This does not make trace volume free. Long-running loops can produce more information than a model should consume directly, so trace slicing, filtering, and semantic queries remain important tooling work. The advantage is that the underlying execution record is deterministic and machine-readable rather than an implicit debugger session.

The compiler as a semantic oracle

Because Moss closes the reachable program before code generation, the compiler already knows facts that an agent would otherwise have to infer approximately from source text: which concrete callable a use

selects, what state a handler may read/write/consume, which concrete domain instance a route names, which handlers conflict, and which synchronization classes a handler acquires. Exposing those facts through semantic queries lets an agent ask the compiler instead of reverse-engineering the compiler.

This is a deliberate design direction, but it should not be confused with changing Moss semantics for AI. The language remains human-readable and its safety arguments do not depend on an AI system. Agent tooling consumes the same checked semantic objects used for native compilation and Fast Debug.

Restrictions can improve generation quality

Several v0.1 restrictions are useful to agents for the same reason they are useful to static analysis. Domain handles cannot flow through arbitrary values; routing is immutable; the concrete domain graph is closed; general dynamic dispatch is absent; Moss-managed synchronization is not programmable; and message boundaries are explicit. These rules remove large families of plausible-but-invalid implementation strategies from the search space.

The claim is deliberately modest: Moss does not make an agent correct, and v0.1 is not presented as an “AI programming language.” Rather, Moss gives an agent a more constrained target language and richer compiler feedback. The expectation is that this can improve first-attempt validity, reduce repair turns, and make debugging less dependent on fragile interactive state. Phase 22 is reserved for measuring and improving those properties after the language has first been dogfooded as a language.

16 Intentional v0.1 boundaries

The following are deliberate v0.1 limits, not accidental omissions hidden by the paper:

- no runtime dynamic dispatch or general first-class closures;
- no ordinary recursion yet (an implementation/language-surface restriction, not a synchronization-proof requirement);
- no dynamic domain creation, first-class domain handles, or topology-affecting runtime control flow;
- no source-level thread/task ingress construct yet;
- no asynchronous domain messaging;
- no programmer-written Moss lock/atomic syntax;
- no finalized Rust interoperability or supervision/error-recovery model;
- no dynamic key-based lock partitioning inside runtime-indexed aggregates.

Concurrent ingress is a deliberate exclusion. v0.1 specifies what is true once control enters a domain handler; it does not yet specify a source-level mechanism that creates independent concurrent roots. The production backend nevertheless assumes concurrent handler entry is possible and may not erase synchronization merely because `main` appears sequential.

17 Future domain scopes: an intentionally open breadcrumb

Current v0.1 composition effectively creates domains for the lifetime of the enclosing program scope. The synchronization plan is intentionally graph-relative rather than process-global so that future work can explore lexical domain scopes and repeated graph activations.

An attractive future possibility is an inner scope that statically imports an outer routing anchor during composition, while preventing any inner handle from escaping upward. However, the rank composition and

repeated-activation semantics are intentionally unresolved. Moss v0.1 does not choose between independent nested subgraphs and nested graphs that may route to longer-lived outer domains.

Formal Model and Safety Arguments

18 Static objects of the model

For a checked graph context G , let D range over concrete domain instances. Each instance has a concrete specialization, a finite set of handlers H_D , and a finite set of state leaves L_D . Route edges form a concrete directed acyclic graph.

The implementation currently plans over the checked handlers present in each concrete specialization rather than performing a whole-program handler-reachability prune first. Consequently, an unreachable declared handler can conservatively inflate the mutable universe and class partition. Reachability pruning is a semantics-preserving future compiler optimization; the proof below does not rely on it.

A deterministic topological linearization assigns each concrete instance a unique $\text{domain_rank}(D)$. If the graph contains edge $D \rightarrow E$, then:

$$\text{domain_rank}(D) < \text{domain_rank}(E).$$

19 State effects and observable effects

For handler $h \in H_D$, static effect analysis derives may-effect sets:

$$R_D(h), \quad W_D(h), \quad C_D(h) \subseteq L_D,$$

representing domain-owned leaves that may be read, written, or consumed on any control-flow path.

The analysis is conservative and currently path-insensitive at the handler-summary level: effects are unioned across branches. If one branch writes a cache entry and another only reads it, the handler may still require exclusive synchronization on every invocation. This is a known conservative bound, not a claim of per-execution minimal locking.

We separately use $O(h)$ for observable non-domain-state effects. O is part of the semantic discussion but does not drive the v0.1 synchronization partition. It is therefore possible for domain-local state to be serializable while independent domains produce externally visible effects in different orders.

Semantic effects on a leaf normalize as:

$$\text{CONSUME} > \text{WRITE} > \text{READ} > \text{NONE}.$$

CONSUMERemains distinct from WRITERfor ownership, even though both require exclusive synchronization.

20 Mutable universe and protected reads

Define the exclusive leaf set for a handler:

$$X_D(h) = W_D(h) \cup C_D(h).$$

The protected mutable universe is:

$$X_D^* = \bigcup_{g \in H_D} X_D(g).$$

Only leaves in X_D^* receive synchronization classes. A leaf outside X_D^* is immutable after publication with respect to the checked handler set and may be read without a runtime domain lock once publication is established.

Protected reads are:

$$\text{ProtectedRead}_D(h) = R_D(h) \cap X_D^*.$$

The leaf-level lock footprint is:

$$\text{LockSet}_D(h) = X_D(h) \cup \text{ProtectedRead}_D(h).$$

21 Synchronization modes and class partition

For synchronization, normalized effects map to modes:

$$\text{CONSUME, WRITE} \mapsto \text{EXCLUSIVE}, \quad \text{READ} \mapsto \text{SHARED}, \quad \text{NONE} \mapsto \text{NONE}.$$

For each leaf $\ell \in X_D^*$, define its synchronization-mode signature over a deterministic ordering of handlers:

$$\sigma_D(\ell) = \langle m(h_1, \ell), \dots, m(h_n, \ell) \rangle.$$

Two leaves belong to the same synchronization class iff their complete signatures are equal. This is a compiler partitioning policy, not source-level semantics or module ABI.

Let $\text{class}_D(\ell)$ map a protected leaf to its class. Then:

$$\text{ClassSet}_D(h) = \{\text{class}_D(\ell) \mid \ell \in \text{LockSet}_D(h)\}.$$

For a class in a handler's ClassSet , the handler mode is **EXCLUSIVE** if the signature is exclusive for that handler, otherwise **SHARED**.

21.1 Class count is a cost as well as an opportunity

The signature partition preserves every statically visible distinction in handler synchronization behavior. It does not optimize a workload-dependent contention objective. More classes can expose more independent concurrency, but each acquired class still costs a native lock acquisition. The baseline partition is therefore contention-oblivious: it favors preserving potential independence rather than minimizing lock count under an observed workload. Profile-guided coarsening or alternative class objectives are future optimization questions, not v0.1 semantics.

22 Running Account derivation

For the **Account** example:

Handler	R	W	C
<code>rename</code>	–	<code>display_name</code>	–
<code>set_risk_limit</code>	–	<code>risk_limit</code>	–
<code>record_fill</code>	<code>balance, stats, risk_limit</code>	<code>balance, stats</code>	–
<code>read_stats</code>	<code>stats</code>	–	–
<code>read_config</code>	<code>config_value</code>	–	–

Table 3: Representative may-effects for the running Account domain.

Thus:

$$X_D^* = \{\text{balance}, \text{stats}, \text{risk_limit}, \text{display_name}\}.$$

`config_value` is outside X_D^* , so `read_config` requires no domain synchronization after safe publication.

The signatures are distinct:

Leaf	Handler modes	Result
<code>balance</code>	<code>record_fill=EXCLUSIVE</code>	class A
<code>stats</code>	<code>record_fill=EXCLUSIVE, read_stats=SHARED</code>	class B
<code>risk_limit</code>	<code>set_risk_limit=EXCLUSIVE, record_fill=SHARED</code>	class C
<code>display_name</code>	<code>rename=EXCLUSIVE</code>	class D

Therefore `record_fill` acquires A and B exclusively and C shared. `rename` is synchronization-disjoint from `record_fill`; `read_stats` can overlap another `read_stats` but conflicts with `record_fill`.

23 Path-insensitive locking: the cache-populate bound

Suppose a cache handler is conceptually:

```
on get(key):
  if !entries.contains(key):
    entries[key] = compute(key)
  reply entries[key]
```

The handler may write `entries`, so its normalized synchronization mode for that class is EXCLUSIVE on every invocation, even when the requested key is already present. Moss v0.1 deliberately accepts this conservative cost in exchange for a complete precomputed ClassSet, no lock upgrade, and simple whole-handler 2PL.

A future path-sensitive design could specialize paths or split handlers, but it would need to preserve the same observable semantics and deadlock argument. The paper does not claim v0.1 produces the weakest lock mode for each dynamic execution.

24 Static-footprint strict two-phase locking

Each handler knows its complete ClassSet before the body begins. It acquires exactly that set in increasing local `class_rank`, then holds every acquired class until terminating reply or normal handler completion. There are no shared-to-exclusive upgrades.

This has the *static-footprint* property often associated with conservative 2PL: the required lock set is known before body execution. However, Moss acquires locks sequentially rather than atomically preclaiming an all-or-none set. Consequently, local class order remains necessary for deadlock freedom.

Theorem 24.1 (Domain-local conflict serializability). *Assume the state-effect summaries are sound, every protected access occurs under the mode prescribed by the handler’s synchronization classes, and locks are held through handler completion. Then failure-free completed handler executions on one domain are conflict-serializable with respect to the domain’s protected state.*

Argument. Every conflicting protected access shares a synchronization class with at least one exclusive mode. RwLock compatibility therefore orders the conflicting accesses. Strict hold-to-completion prevents a handler from exposing a partial protected-state serialization point and later reacquiring a conflicting class. Standard 2PL conflict-serializability reasoning applies to the protected domain state [3].

This theorem is intentionally local to protected state. It does not make a nested multi-domain call tree into an atomic transaction, nor does it impose one total order on $O(h)$ across independent domains.

25 Global lock rank and deadlock freedom

Each synchronization class has a deterministic local `class_rankD(C)`. Define:

$$\text{LockRank}(D, C) = (\text{domain_rank}(D), \text{class_rank}_D(C))$$

with lexicographic order.

Within a handler, classes are acquired in increasing class rank. Across a nested `message`, the target lies on an outgoing concrete route edge and therefore has greater domain rank. Parent locks remain held while the child acquires its classes.

Theorem 25.1 (Deadlock freedom for Moss-managed domain locks). *Under the closed acyclic concrete routing graph, deterministic increasing local class acquisition, no lock upgrades, and nested messages only along declared route edges, Moss-managed domain locks cannot participate in a wait-for cycle.*

Proof sketch. Every newly acquired Moss-managed lock has strictly greater LockRank than every Moss-managed lock still held by the current thread. A deadlock cycle would imply

$$L_1 < L_2 < \dots < L_n < L_1,$$

which is impossible for a strict total order. Within one domain, local class rank is sufficient; across nesting, domain rank is the load-bearing component. The ordering applies to currently held locks, not historical acquisitions, so a caller may return from a higher-ranked sibling and then enter a lower-ranked sibling provided that sibling still ranks above the held ancestor locks.

26 Why the concrete graph must be closed

The deadlock proof depends on every legal cross-domain message target appearing in the concrete route graph. That is why domain handles are not first-class values. If a handler could receive a domain handle in a payload and later call it, runtime execution could introduce a lock-acquisition edge that the static topology and rank assignment never saw.

Moss therefore restricts handles to concrete composition bindings, declared route bindings, and message receivers. This is not merely an ergonomic restriction; it is part of the proof boundary.

27 Publication and immutable reads

Construction and publication must satisfy a happens-before condition:

All initialization writes to a concrete domain instance happen-before any concurrent handler execution that may observe that instance.

This formulation is stronger and more precise than saying merely that “construction finishes first.” It is the reason leaves outside X_D^* may be read without a domain lock: the compiler has established that checked handlers never mutate them, while the runtime establishes safe publication of their initialized values.

28 Ownership, aliases, and domain boundaries

Ordinary calls may preserve caller storage identity according to inferred parameter effects. The compiler checks read/write/consume capabilities and rejects overlapping aliases when one side may write or consume.

Domain boundaries deliberately break ordinary alias identity. A message argument establishes an independent semantic value; so does a reply result. The backend may optimize the physical transfer when it can prove observational equivalence, but cross-domain mutable aliases are not part of the language model.

29 Failure model and proof scope

Unexpected failure semantics are not yet a language-level supervision model. v0.1 fails closed: the production backend aborts on unexpected handler failure or poisoned synchronization rather than releasing possibly inconsistent state and continuing.

All serializability and normal-exit state-validity arguments in this paper are scoped to failure-free completed handlers. Phase 21 is reserved for explicit error propagation and supervision semantics.

30 Claims and non-claims

Claims

- Safe Rust remains the physical memory-safety substrate for generated production code.
- Moss statically closes concrete domain routing in v0.1.
- Moss derives domain-state synchronization from specialized may-effects rather than user-written locks.
- Protected domain state is conflict-serializable under the stated failure-free assumptions.
- Moss-managed domain lock deadlock is structurally excluded under the closed-graph/rank assumptions.
- Message/reply boundaries are semantically by value and do not expose cross-domain aliases.

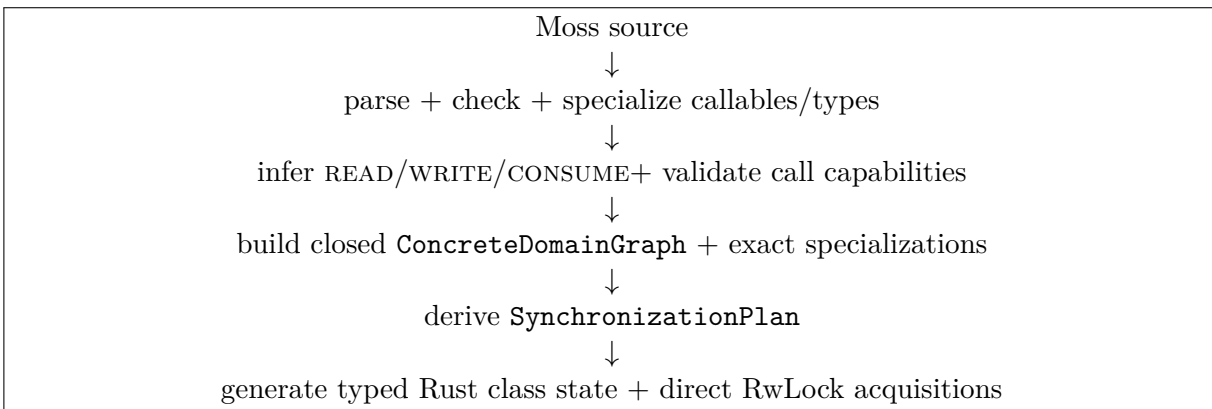
Non-claims

- Moss does not improve on every safety property of Rust; it is more restrictive and obtains additional guarantees in selected concurrency dimensions.
- Moss does not promise global sequential consistency across the entire domain graph.
- Moss does not provide one transaction spanning descendant domains or arbitrary external effects.
- Moss does not guarantee fairness, starvation freedom, lock-free progress, or bounded waiting.
- v0.1 does not define a source-level concurrent-ingress mechanism.
- v0.1 does not yet define supervision, rollback, restart, or recovery after unexpected handler failure.
- The deadlock proof covers Moss-managed locks, not arbitrary future foreign locks acquired invisibly by external code.
- The synchronization partition is not guaranteed workload-optimal or path-minimal.

Implementation Design, Corrections, and Measurements

31 Compilation pipeline

The final v0.1 pipeline is conceptually:



The design deliberately keeps semantic artifacts separate from physical lowering. Module interfaces carry checked semantic information; the final application generates its own synchronization partition and physical Rust layout.

32 Final production lowering: locks own their protected state

The final backend emits one typed Rust state structure per synchronization class. A representative shape is:

```

struct AccountClass0 { balance: i64 }
struct AccountClass1 { stats: Stats }
struct AccountClass2 { risk_limit: i64 }
struct AccountClass3 { display_name: String }
struct AccountImmutable { config_value: i64 }

struct AccountRuntime {
    class0: RwLock<AccountClass0>,
    class1: RwLock<AccountClass1>,
    class2: RwLock<AccountClass2>,
    class3: RwLock<AccountClass3>,
    immutable: AccountImmutable,
}
  
```

A handler emits direct, statically known acquisitions. There is no runtime handler-plan interpretation:

```

let mut c0 = moss_write_or_abort(&self.state.class0);
  
```

```

let mut c1 = moss_write_or_abort(&self.state.class1);
let c2 = moss_read_or_abort(&self.state.class2);

let mut view = RecordFillView {
    balance: &mut c0.balance,
    stats: &mut c1.stats,
    risk_limit: &c2.risk_limit,
};

let result = __moss_body_record_fill(&mut view, fill);
result

```

Rust RAII retains the guards for the handler scope. Nested messages run while parent guards remain live.

33 Why the first generic runtime was rejected

The first correct 2PL backend interpreted synchronization metadata at runtime using handler descriptors, class/leaf maps, guard maps, and generic take/restore scaffolding. It was safe and useful while the architecture was changing, but Phase 10.6F measured avoidable fixed overhead on tiny handlers.

That overhead was classified as a v0.1 blocker because the compiler already knew every class, mode, and leaf statically. Phase 10.6F.1 replaced runtime plan interpretation with static typed lowering. The result is an important implementation lesson: *the compiler should execute the planning work at compile time and emit ordinary direct Rust, not carry a generic synchronization interpreter into the production hot path.*

34 Borrowed protected reads

A second correction removed hidden deep copies of protected READ state. Protected reads now borrow directly from the data owned by the retained shared guard; immutable-after-publication reads borrow directly from immutable storage. Whole and nested objects are represented with typed borrowed views when needed.

Message and reply boundaries still materialize independent values. The optimization therefore preserves the language-level distinction:

$$\text{ordinary READ} \Rightarrow \text{borrow}, \quad \text{message/reply} \Rightarrow \text{independent semantic value}.$$

No unsafe Rust was introduced for the final v0.1 backend.

35 From asynchronous actors to synchronous domains

The design began with actor-like mailboxes, queues, worker loops, **await**, sender FIFO, and a per-domain total commit-order concept. That machinery became unnecessary once shared-memory **message** was made synchronous.

The final model removed **await**, mailboxes, queues, worker threads, sender FIFO semantics, commit-order machinery, alternate atomic-domain lowering, batching, coalescing, and domain-cluster execution. This simplification sharpened the actual abstraction: a domain is protected state with statically known routing, not a promise of independent scheduling.

36 Closing the domain-handle universe

Static routing was initially incomplete because domain handles could still behave like values in some paths. Phase 10.6B.1 closed that hole: handles cannot cross message/reply, ordinary parameters, aggregates, or mutable state. Every legal cross-domain call is now statically attributable to a route edge.

This implementation correction was proof-relevant rather than cosmetic. Without it, `domain_rank` would not describe the actual nested lock-acquisition graph.

37 Fast Debug as a semantic engine

Fast Debug interprets the already-checked program. It constructs logical concrete domain instances, follows checked routes, executes messages as nested interpreter frames, mutates logical state, and terminates on reply. It intentionally does not simulate `RwLocks`, contention, or thread interleavings.

This creates a useful separation:

	Production	Fast Debug
Semantics	Checked Moss semantics	Same checked Moss semantics
Concurrency	May receive concurrent handler entry	One deterministic legal schedule
Locks	Physical compiler-derived <code>RwLocks</code>	None
Purpose	Native execution	Fast semantic debugging/tracing

38 Performance validation: codegen overhead, not a scalability claim

The strongest v0.1 performance evidence answers a narrow implementation question: after static typed lowering, does a known Moss lock footprint cost materially more than equivalent handwritten Rust using the same `std::sync::RwLock`? On the Phase 10.6F.1 environment, the answer was no for the one-to-three-lock microcases.

Case	Typed Moss (ns/call)	Hand Rust (ns/call)	Ratio
1 SHARED	11.782	11.652	1.011
1 EXCLUSIVE	11.647	11.613	1.003
2 EXCLUSIVE	20.947	20.301	1.032
Mixed 2-class	21.639	21.171	1.022
3-class	30.120	29.382	1.025

Table 4: Phase 10.6F.1 tight-loop medians. These validate codegen overhead, not application scalability.

The measurements used the same `RwLock` primitive and equivalent logical work. Optimized assembly inspection found no map operations, string comparisons, descriptor loops, heap allocator calls, or Arc refcount increments on the protected hot paths. The remaining small differences are consistent with ordinary call/codegen and fail-closed machinery rather than synchronization-plan interpretation [14].

38.1 What the concurrency measurements do not prove

The broader threaded measurements were run on an Intel i3-1315U environment with mixed core types and scheduler/frequency noise. They are useful sanity checks—shared readers overlapped, disjoint writer classes overlapped, conflicting writers serialized—but they are **not evidence of general scalability**.

A submission-quality scalability study should use uniform physical cores, affinity/pinning, repeated distributions, and workloads that isolate class partitioning from ordinary code-generation effects.

The current paper therefore treats the threaded measurements as implementation validation rather than a headline performance result.

39 Class count, nested hold time, and other measured costs

More synchronization classes expose more potential parallelism but also mean more native lock acquisitions. The v0.1 signature partition is intentionally workload-agnostic. Dogfooding may reveal domains where class coarsening would outperform maximal static separation under realistic contention.

Nested synchronous messages also lengthen ancestor critical sections because parent guards remain held while descendants run. Instrumented Phase 10 measurements found descendant time dominating ancestor hold time in a representative nested sample. That behavior is expected under the baseline proof and is documented rather than hidden. Early unlock would require a separate observational-equivalence argument and is not part of v0.1.

40 Modules: semantic ABI, physical lowering at the consumer

Source-free providers export semantic information needed for downstream specialization and effect analysis. The final consumer builds the concrete graph, derives synchronization, and emits typed lock-owned state. Physical classes and Rust lifetimes are deliberately absent from the semantic module ABI.

This architecture matters because it lets the final application specialize imported behavior without forcing a provider to predict the consumer’s concrete domain instances or lock partition.

41 Reachability and other conservative bounds

The current synchronization plan includes checked handlers in the concrete specialization even if whole-program analysis could prove some are unreachable. Such a handler can enlarge X_D^* and split/introduce synchronization classes unnecessarily. This is a straightforward optimization opportunity: prune unreachable handlers before constructing the partition, while preserving diagnostics for declared but unused code.

Other conservative bounds include path-insensitive may-effects and coarse treatment of runtime-indexed aggregates. These are engineering tradeoffs in v0.1, not weaknesses in the deadlock argument.

42 Design evolution summary

Stage	Earlier idea	v0.1 resolution
Domain execution	Mailbox, queue, worker, <code>await</code>	Synchronous direct message
Ordering	Per-domain total commit order	Domain-local conflict serializability; no graph-wide total order
Topology	Handles could still flow dynamically	Closed static concrete routing graph
Synchronization	Coarse/alternate backends	Compiler-derived classes + handler-level 2PL
READ lowering	Hidden snapshots/clones	Borrowed protected views

Stage	Earlier idea	v0.1 resolution
Physical runtime	Generic maps/descriptors	Static typed <code>RwLock<ClassState></code> lowering
Traits	Potential nominal membership	Structural compile-time predicates; no <code>implements</code>
Parameter mutation	Considered explicit modifiers	Inferred <code>READ/WRITE/CONSUME</code> effects
Debugging	Native debugger required for domains	Deterministic Fast Debug semantic engine
Language growth	Continue adding features	Phase 15 dogfooding before expansion

43 Dogfooding before expansion

Phase 10 defines the Moss v0.1 usable-language milestone. The next milestone is Phase 15: dogfooding. The goal is to write real programs and let concrete friction drive subsequent work.

Already-known questions include ordinary recursion, module/package ergonomics, trace slicing, source-free generic ergonomics, domain lifetime scopes, and standard-library gaps. None should be solved merely because the roadmap has room. The language should now earn its next features through use.

Related Work and Positioning

44 Synchronization inference and lock allocation

Compiler-assisted lock inference is direct prior art. Autolocker infers synchronization for atomic sections [5]. Emmi et al.'s lock allocation infers lock assignments and instrumentation intended to preserve atomicity and deadlock freedom [6]. Cherem, Chilimbi, and Gulwani infer locks for atomic sections using program analysis [7].

Moss should therefore not claim novelty merely because the compiler derives locks. The distinction is the surrounding language contract. Moss does not start from arbitrary shared-memory code annotated with atomic regions. It starts from statically composed source-level domains, closes the *concrete instance* routing graph, derives effects after specialization, partitions domain-owned state from exact concrete handler signatures, and uses the same instance graph to establish inter-domain lock rank. Domain handles cannot escape and create unseen runtime edges. In that sense, the synchronization inference is one component of a deliberately restricted language rather than a retrofit onto unrestricted shared memory.

45 Effect systems and Deterministic Parallel Java

Effect systems provide a long history of static reasoning about program behavior [4]. Deterministic Parallel Java (DPJ) uses a type-and-effect system to provide strong compile-time guarantees for deterministic parallel programming [8]. Moss shares the idea that effects can turn concurrency properties into compile-time facts, but pursues a different user model: structural specialization, source-level state-owning domains, compiler-derived lock partitioning, and static concrete routing rather than programmer-managed regions as the primary architecture.

The important comparison is not “effects versus no effects,” but how much of the concurrency structure the language makes statically recoverable from ordinary source.

46 Actors and capability-based race freedom

Actors traditionally isolate mutable state behind asynchronous message processing [10, 11]. Pony’s deny capabilities show how a carefully designed capability system can support race-free actor programming with strong static properties [9].

Moss deliberately diverges from classical actor scheduling. A Moss domain is not a mailbox/worker abstraction, messages are synchronous, and multiple compatible handlers on one domain may overlap under compiler-derived shared-memory synchronization. The common idea is to make ownership boundaries explicit enough for the language to reason about concurrency; the execution model is different.

47 Rust and Julia as complementary influences

Rust supplies the physical safety substrate and a reference point for systems-programming guarantees [2]. Moss’s goal is not to weaken Rust’s model, but to make a narrower concurrency discipline easier to use by removing lock selection and order from application source.

Julia supplies a different influence: concise source, specialization, and the idea that high-level generic code need not imply slow generic execution [1]. Moss borrows that design attitude while statically closing behavior that Julia may leave to runtime dispatch. The resulting combination is intentionally asymmetric: Julia-like ergonomic ambition at the source level, Rust-backed safety/native execution at the implementation level, and additional compile-time restrictions around Moss-managed concurrency.

48 Open questions

The v0.1 paper leaves several questions deliberately open:

- whether ordinary recursion should be introduced once specialization termination and diagnostics are designed;
- how lexical domain scopes should compose with outer routing anchors and repeated activations;
- whether profile-guided synchronization-class coarsening is worthwhile;
- how to add concurrent ingress without corrupting the static domain model;
- how Rust interoperability should constrain foreign aliases and foreign lock acquisition;
- how supervision/restart semantics interact with state validity and synchronous message chains;
- which module/package improvements are justified by Phase 15 dogfooding.

49 Conclusion

Moss’s central wager is that a language can feel lighter by making the compiler responsible for facts programmers usually restate manually. The surface aims for Julia-like economy: structural constraints, specialization, and aggressive inference. The implementation deliberately leans on Rust rather than rebuilding a memory-safe native backend. And the concurrency model narrows the space further so that domains, routes, effects, synchronization classes, and lock order are statically established before code generation.

The result is not “Rust but safer” in every dimension, nor “Julia but static.” It is a deliberately restricted point in the design space: high-level source, static closure, safe-Rust lowering, and compiler-owned synchronization. The v0.1 implementation experience suggests that the restrictions are strong enough to support useful proofs and that those proofs need not require an expensive runtime abstraction: after static typed lowering, one-to-three-lock handler wrappers measured in the same cost class as equivalent handwritten Rust.

The next test is not another architecture phase. It is whether Moss is pleasant enough to use on real programs that its restrictions feel like leverage rather than friction.

A Compact v0.1 semantic summary

Area	v0.1 rule
Typing	Static specialization; untyped parameters are statically specialized duck typing
Traits	Structural compile-time predicates; no nominal <code>implements</code> requirement
Callable dispatch	Statically resolved; no runtime dynamic dispatch
General closures	Not first-class in v0.1; restricted specialized functional captures exist
Ordinary recursion	Disallowed in v0.1 for implementation/specialization reasons, not the synchronization proof
Ownership effects	READ/WRITE/CONSUME inferred and call-site capability checked
Domains	Static state-owning architectural/synchronization units
Routes	Immutable <code>domainroutes</code> ; concrete graph closed and acyclic
Domain handles	Routing capabilities, not ordinary values
Message	Synchronous, blocking, expression-valued
Reply	Terminating, by-value semantic result
Payloads	Incoming snapshots immutable; forwarding/reply creates new value boundaries
Consistency	Domain-local protected-state conflict serializability; no global SC across independent domains
External effects	Ordered by ordinary synchronous program dependencies; no global total order
Synchronization	Compiler-derived classes; exact handler <code>ClassSet</code> known before body
Locking	Static-footprint strict 2PL; no upgrades; full-handler hold
Deadlock order	Lexicographic (<i>domain_rank</i> , <i>class_rank</i>) for currently held Moss locks
Physical backend	Static typed safe Rust, <code>RwLock<ClassState></code> per synchronization class
Fast Debug	Deterministic semantic interpreter; no lock/thread simulation
Ingress	Source-level concurrent root creation intentionally deferred
Failure	Unexpected failure is fail-closed; supervision deferred

B Proof assumptions checklist

The formal claims rely on the following conditions:

1. Static effect analysis conservatively contains every relevant handler state read/write/consume, including specialized helper and capture effects.
2. Concrete domain routes are closed; legal messages cannot introduce unseen targets.

3. The concrete route graph is acyclic and deterministically topologically ranked.
4. Every protected leaf belongs to exactly one synchronization class derived from its complete handler-mode signature.
5. Every handler acquires exactly its ClassSet in increasing class rank before executing protected accesses.
6. No shared-to-exclusive upgrade occurs.
7. Parent locks remain held across nested synchronous messages.
8. Every nested message follows a route to a greater domain rank.
9. Initialization writes happen-before concurrent handler execution observes the domain instance.
10. Message/reply value boundaries do not leak mutable aliases between domains.
11. Normal handler exit leaves every domain-owned value ownership-valid.
12. Serializability claims are for failure-free completed handlers; unexpected failure is fail-closed in v0.1.
13. Foreign code does not acquire hidden Moss-state aliases or violate Moss-managed lock-order assumptions; detailed interop remains future work.

C Moss v0.1 milestone map

Milestone	Result
Phase 10	Moss v0.1 usable-language milestone
10.5	Semantic convergence, stable identities, effects/introspection groundwork
10.6A	Synchronous message , terminating reply , await retired
10.6B	Static composition, domainroutes , concrete DAG, domain rank
10.6B.1	Closed handle universe and exact concrete specialization linkage
10.6C	Compiler-owned SynchronizationPlan
10.6D	Production handler-level 2PL
10.6D.1	Borrowed protected reads; hidden READ snapshots removed
10.6E	Legacy runtime retired; Fast Debug domains aligned
10.6F	Diagnostics, metrics, implementation validation
10.6F.1	Static typed synchronization lowering; runtime plan interpretation removed
Phase 15	Dogfooding: write real Moss programs before speculative expansion
Phase 20	Rust interoperability
Phase 21	Error propagation and supervision
Phase 22	Agent agency tooling

D References

References

- [1] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah. Julia: A Fresh Approach to Numerical Computing. *SIAM Review*, 59(1):65–98, 2017. <https://doi.org/10.1137/141000671>.
- [2] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. RustBelt: Securing the Foundations of the Rust Programming Language. *Proceedings of POPL*, 2018.
- [3] K. P. Eswaran, J. N. Gray, R. A. Lorie, and I. L. Traiger. The notions of consistency and predicate locks in a database system. *Communications of the ACM*, 19(11):624–633, 1976.

- [4] J. M. Lucassen and D. K. Gifford. Polymorphic effect systems. *Proceedings of POPL*, pages 47–57, 1988.
- [5] B. McCloskey, F. Zhou, D. Gay, and E. A. Brewer. Autolocker: Synchronization Inference for Atomic Sections. *Proceedings of POPL*, 2006.
- [6] M. Emmi, J. Fischer, R. Jhala, and R. Majumdar. Lock Allocation. *Proceedings of POPL*, 2007. <https://doi.org/10.1145/1190215.1190260>.
- [7] S. Cherem, T. M. Chilimbi, and S. Gulwani. Inferring Locks for Atomic Sections. *Proceedings of PLDI*, pages 304–315, 2008. <https://doi.org/10.1145/1375581.1375619>.
- [8] R. L. Bocchino Jr. et al. A Type and Effect System for Deterministic Parallel Java. *Proceedings of OOPSLA*, pages 97–116, 2009. <https://doi.org/10.1145/1640089.1640097>.
- [9] S. Clebsch, S. Drossopoulou, S. Blessing, and A. McNeil. Deny Capabilities for Safe, Fast Actors. *Proceedings of AGERE! 2015*, 2015.
- [10] C. Hewitt, P. Bishop, and R. Steiger. A Universal Modular ACTOR Formalism for Artificial Intelligence. *Proceedings of IJCAI*, pages 235–245, 1973.
- [11] G. A. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.
- [12] Chapel Project. Chapel language documentation: arrays and domains. <https://chapel-lang.org/>.
- [13] Moss Project. Moss v0.1 implementation and documentation, 2026. <https://github.com/kuttyb/moss>.
- [14] Moss Project. Phase 10.6F.1 static typed synchronization validation, 2026. https://github.com/kuttyb/moss/blob/main/docs/PERFORMANCE_10_6F_1.md.